Development Environment Setup Guide

Table of Contents

1. Prerequisites
2. Initial Setup
3. Environment Configuration
4. Database Setup
5. External API Configuration
6. Running the Application
7. Development Tools
8. Troubleshooting
9. Next Steps

Prerequisites

Required Software

1. Node.js and npm

- Version: Node.js 18.x or higher
- Check version: `node --version`
- Download: https://nodejs.org/

2. Python (for backend services)

- Version: Python 3.10 or higher
- Check version: `python --version` or `python3 --version`
- Download: https://www.python.org/downloads/

3. Git

- Check version: `git --version`
- Download: https://git-scm.com/downloads

4. Docker and Docker Compose (recommended)

- Docker Desktop includes both
- Check version: `docker --version` and `docker-compose --version`
- Download: https://www.docker.com/products/docker-desktop

5. PostgreSQL (if not using Docker)

- Version: 14 or higher
- Download: https://www.postgresql.org/download/

6. Redis (if not using Docker)

- Version: 6.x or higher
- Download: https://redis.io/download

Optional but Recommended

- **VS Code** with extensions:
  - ESLint
  - Prettier
  - Python
  - Docker
  - GitLens
- **Postman** or **Insomnia** for API testing
- **pgAdmin** or **DBeaver** for database management

System Requirements

- **OS**: macOS, Linux, or Windows 10/11
- **RAM**: Minimum 8GB (16GB recommended)
- **Storage**: At least 10GB free space
- **Internet**: Required for downloading dependencies and API calls

---

Initial Setup

1. Clone the Repository

bash

```
# Clone the repository
git clone https://github.com/yourusername/ai-gaming-companion.git

# Navigate to project directory
cd ai-gaming-companion

# Create a new branch for your development
git checkout -b dev/your-name
```

2. Install Dependencies

Frontend (if applicable)

bash

```
# Navigate to frontend directory
cd frontend

# Install npm packages
npm install

# Return to root
cd ..
```

Backend

bash

```
# Navigate to backend directory
cd backend

# Create virtual environment
python -m venv venv

# Activate virtual environment
# On macOS/Linux:
source venv/bin/activate
# On Windows:
venv\Scripts\activate

# Install Python dependencies
pip install -r requirements.txt

# Return to root
cd ..
```

Alternative: Install Everything from Root

```
bash

# If you have a unified package structure
npm install

# For Python dependencies
pip install -r requirements.txt
```

Environment Configuration

1. Copy Environment Template

```
bash

# Copy the example environment file
cp .env.example .env
```

2. Configure Environment Variables

Open `.env` in your text editor and configure the following:

Application Settings

bash

```
# Application
NODE_ENV=development
PORT=3000
API_PORT=8000

# Application URLs
FRONTEND_URL=http://localhost:3000
BACKEND_URL=http://localhost:8000

# Secret Keys (generate secure random strings)
JWT_SECRET=your-super-secret-jwt-key-change-this
SESSION_SECRET=your-super-secret-session-key-change-this
ENCRYPTION_KEY=your-32-character-encryption-key
```

Generating Secure Keys:

bash

```
# On macOS/Linux, generate random strings:
openssl rand -base64 32

# Or use Node.js:
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
```

Database Configuration

bash

```
# PostgreSQL
DATABASE_URL=postgresql://username:password@localhost:5432/gaming_companion
DB_HOST=localhost
DB_PORT=5432
DB_NAME=gaming_companion
DB_USER=your_username
DB_PASSWORD=your_password

# Redis
REDIS_URL=redis://localhost:6379
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=  # Leave empty for local development
```

AI/LLM Configuration

bash

```
# Anthropic Claude
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxx
CLAUDE_MODEL=claude-sonnet-4-20250514
CLAUDE_MAX_TOKENS=4000

# OpenAI (optional, if using GPT)
OPENAI_API_KEY=sk-xxxxxxxxxxxx
OPENAI_MODEL=gpt-4-turbo-preview

# LLM Settings
LLM_TEMPERATURE=0.7
LLM_TIMEOUT=30000  # milliseconds
```

Game API Configuration

bash

```
# Riot Games API (for League of Legends, Valorant, etc.)
RIOT_API_KEY=RGAPI-xxxxxxxxxxxx
RIOT_API_REGION=na1  # or your region: euw1, kr, etc.

# Steam API (if supporting Steam games)
STEAM_API_KEY=xxxxxxxxxxxx

# Other game APIs as needed
# Add more game API keys here
```

External Services

bash

```
# Web Search (for research agent)
SEARCH_API_KEY=xxxxxxxxxxxx  # Google Custom Search or similar
SEARCH_ENGINE_ID=xxxxxxxxxxxx

# Analytics (optional)
MIXPANEL_TOKEN=xxxxxxxxxxxx
SENTRY_DSN=xxxxxxxxxxxx

# Email Service (for notifications)
SENDGRID_API_KEY=xxxxxxxxxxxx
EMAIL_FROM=noreply@yourdomain.com
```

Storage Configuration

bash

```
# File Storage (AWS S3 or local)
STORAGE_TYPE=local  # or 's3'
STORAGE_PATH=./uploads

# AWS S3 (if using)
AWS_ACCESS_KEY_ID=xxxxxxxxxxxx
AWS_SECRET_ACCESS_KEY=xxxxxxxxxxxx
AWS_REGION=us-east-1
AWS_S3_BUCKET=gaming-companion-uploads
```

Rate Limiting & Cost Management

```bash
# Rate Limits
RATE_LIMIT_WINDOW_MS=900000  # 15 minutes
RATE_LIMIT_MAX_REQUESTS=100

# Cost Management
MAX_MONTHLY_COST_FREE=0.50
MAX_MONTHLY_COST_BASIC=5.00
MAX_MONTHLY_COST_PRO=25.00
```

Database Setup

Option 1: Using Docker (Recommended)

1. Start Database Services

```bash
# Start PostgreSQL and Redis using Docker Compose
docker-compose up -d postgres redis

# Verify services are running
docker-compose ps
```

2. Create Database

The database is automatically created by Docker Compose. Skip to Run Migrations.

Option 2: Local Installation

1. Start PostgreSQL

```bash
# macOS (with Homebrew)
brew services start postgresql@14

# Linux
sudo systemctl start postgresql

# Windows - use PostgreSQL service manager
```

2. Create Database

```bash
# Connect to PostgreSQL
psql -U postgres

# Create database
CREATE DATABASE gaming_companion;

# Create user (optional, if not using default)
CREATE USER gaming_user WITH PASSWORD 'your_password';

# Grant privileges
GRANT ALL PRIVILEGES ON DATABASE gaming_companion TO gaming_user;

# Exit psql
\q
```

3. Start Redis

```bash
# macOS (with Homebrew)
brew services start redis

# Linux
sudo systemctl start redis

# Windows - use Redis service or run redis-server.exe
```

3. Run Migrations

```bash
```

```
# Navigate to backend directory
cd backend

# Run database migrations
npm run migrate
# or
python manage.py migrate

# Seed initial data (optional)
npm run seed
# or
python manage.py seed
```

4. Verify Database Connection

```
bash

# Test database connection
npm run db:test
# or
python manage.py check_db

# Expected output: "Database connection successful"
```

External API Configuration

1. Anthropic Claude API

1. Go to https://console.anthropic.com/
2. Sign up or log in
3. Navigate to API Keys
4. Create a new API key
5. Copy the key to `.env` as `ANTHROPIC_API_KEY`

Important: Claude API requires billing to be set up. Start with a small credit ($5-10 for testing).

2. Riot Games API (for League of Legends/Valorant)

1. Go to https://developer.riotgames.com/

2. Sign in with your Riot account
3. Register your application
4. Generate a development API key
5. Copy the key to `.env` as `RIOT_API_KEY`

Note: Development keys expire after 24 hours. For production, apply for a production key.

3. Steam API (Optional)

1. Go to https://steamcommunity.com/dev/apikey
2. Register your domain (use localhost for development)
3. Copy the key to `.env` as `STEAM_API_KEY`

4. Web Search API (Optional)

For the research agent, you'll need a search API:

Google Custom Search:

1. Go to https://console.cloud.google.com/
2. Create a new project
3. Enable Custom Search API
4. Create credentials (API Key)
5. Set up a Custom Search Engine at https://cse.google.com/

API Key Security Best Practices

- **Never commit** `.env` file to Git
- Add `.env` to `.gitignore` (should already be there)
- Use different API keys for development and production
- Rotate keys regularly
- Set up spending alerts for paid APIs
- Use environment-specific keys

Running the Application

Full Stack with Docker (Recommended for Beginners)

```bash
# Start all services
docker-compose up

# Or run in background
docker-compose up -d

# View logs
docker-compose logs -f

# Stop all services
docker-compose down
```

Services will be available at:

- Frontend: http://localhost:3000
- Backend API: http://localhost:8000
- PostgreSQL: localhost:5432
- Redis: localhost:6379

Manual Setup (Advanced)

1. Start Backend

```bash
```

```
# Navigate to backend directory
cd backend

# Activate virtual environment (if using Python)
source venv/bin/activate  # macOS/Linux
# or
venv\Scripts\activate  # Windows

# Start development server
npm run dev
# or
python manage.py runserver
# or
uvicorn main:app --reload --port 8000
```

Backend should be running at http://localhost:8000

2. Start Frontend (in new terminal)

```
bash

# Navigate to frontend directory
cd frontend

# Start development server
npm run dev
# or
npm start
```

Frontend should be running at http://localhost:3000

3. Start Worker Queue (in new terminal)

For background tasks and agent processing:

```
bash
```

```
# Navigate to backend directory
cd backend

# Activate virtual environment
source venv/bin/activate

# Start worker
npm run worker
# or
celery -A app worker --loglevel=info
```

Verify Everything is Running

bash

```
# Check backend health
curl http://localhost:8000/health

# Expected response:
# {"status": "healthy", "services": {"database": "up", "redis": "up"}}

# Check frontend
curl http://localhost:3000

# Should return HTML
```

Development Tools

Database Management

Using pgAdmin (GUI)

1. Open pgAdmin
2. Add New Server
   - Host: localhost
   - Port: 5432
   - Database: gaming_companion
   - Username: your_username

- Password: your_password

Using Command Line

```bash
# Connect to database
psql -U your_username -d gaming_companion

# Useful commands:
\dt             # List tables
\d table_name   # Describe table
\q              # Quit
```

Redis Management

Using Redis CLI

```bash
# Connect to Redis
redis-cli

# Useful commands:
KEYS *          # List all keys
GET key_name    # Get value
FLUSHALL        # Clear all data (careful!)
MONITOR         # Watch commands in real-time
```

Using Redis GUI

- **RedisInsight**: https://redis.com/redis-enterprise/redis-insight/
- **TablePlus**: https://tableplus.com/

API Testing

Create a test request file

Create `api-tests/test-agent.http` (works with REST Client extension in VS Code):

```http
```

```
### Health Check
GET http://localhost:8000/health

### Test Agent - Simple Query
POST http://localhost:8000/api/agent/query
Content-Type: application/json

{
  "query": "What's the best build for ADC in League of Legends?",
  "user_id": "test_user_123",
  "fast_response": true
}

### Test Agent - Deep Analysis
POST http://localhost:8000/api/agent/analyze
Content-Type: application/json

{
  "task_type": "match_analysis",
  "user_id": "test_user_123",
  "parameters": {
    "game": "league_of_legends",
    "num_matches": 5
  }
}
```

Monitoring Logs

Docker Logs

bash

```
# View all logs
docker-compose logs -f

# View specific service
docker-compose logs -f backend

# View last 100 lines
docker-compose logs --tail=100 backend
```

Application Logs

```bash
# Backend logs (if running manually)
tail -f backend/logs/app.log

# Frontend logs
# Visible in terminal where npm start was run
```

Troubleshooting

Common Issues

Port Already in Use

```bash
# Find process using port
lsof -i :3000  # macOS/Linux
netstat -ano | findstr :3000  # Windows

# Kill process
kill -9 <PID>  # macOS/Linux
taskkill /PID <PID> /F  # Windows
```

Database Connection Failed

Symptoms: `ECONNREFUSED` or `connection refused`

Solutions:

1. Verify PostgreSQL is running: `docker-compose ps` or `brew services list`
2. Check credentials in `.env` match database
3. Verify port 5432 is not blocked
4. Try connecting manually: `psql -U your_username -d gaming_companion`

Redis Connection Failed

Solutions:

1. Verify Redis is running: `redis-cli ping` (should return `PONG`)
2. Check Redis port in `.env` (default: 6379)

3. Restart Redis: `docker-compose restart redis`

API Key Errors

Symptoms: `401 Unauthorized` or `Invalid API key`

Solutions:

1. Verify API keys in `.env` are correct
2. Check for extra spaces or quotes around keys
3. Ensure API key has not expired (Riot dev keys expire daily)
4. Verify API has billing enabled (for Claude API)

Module Not Found Errors

```bash
# Clear npm cache
npm cache clean --force

# Delete node_modules and reinstall
rm -rf node_modules package-lock.json
npm install

# For Python
pip cache purge
pip install -r requirements.txt --force-reinstall
```

Docker Issues

```bash
# Reset Docker environment
docker-compose down -v  # Removes volumes (will delete data!)
docker-compose up --build

# Check Docker disk space
docker system df
docker system prune  # Clean up unused resources
```

Migration Errors

```bash
# Reset migrations (CAUTION: loses data)
npm run migrate:reset
# or
python manage.py migrate:reset

# Create fresh migration
npm run migrate:fresh
```

Getting Help

If you're stuck:

1. Check the GitHub Issues
2. Review logs for error messages
3. Consult documentation in `/docs` folder
4. Ask in project Discord/Slack (if available)
5. Create a new issue with:
   - Error message
   - Steps to reproduce
   - Your environment (OS, Node version, etc.)

Debug Mode

Enable verbose logging:

```bash
# In .env
LOG_LEVEL=debug
DEBUG=true

# Restart application
```

Next Steps

1. Verify Setup

Run the verification script:

```bash
npm run verify-setup
```

This checks:

- ✓ All dependencies installed
- ✓ Database connection
- ✓ Redis connection
- ✓ API keys configured
- ✓ Environment variables set

2. Run Tests

```bash
# Backend tests
cd backend
npm test
# or
pytest

# Frontend tests
cd frontend
npm test
```

3. Load Sample Data

```bash
# Load test users and sample game data
npm run seed:dev
```

4. Try the Demo

```bash
```

```
# Run interactive demo
npm run demo
```

5. Read Additional Documentation

- Project Overview
- Technical Specification
- API Documentation
- Architecture Guide

6. Start Developing!

bash

```
# Create a new feature branch
git checkout -b feature/your-feature-name

# Make your changes
# ...

# Commit your changes
git add .
git commit -m "feat: add your feature description"

# Push to remote
git push origin feature/your-feature-name
```

Development Workflow

Daily Routine

bash

```
# 1. Pull latest changes
git pull origin main

# 2. Start services
docker-compose up -d

# 3. Start development servers
npm run dev

# 4. Make your changes
# ...

# 5. Run tests before committing
npm test

# 6. Commit and push
git add .
git commit -m "your message"
git push
```

Code Quality

```
bash

# Run linter
npm run lint

# Auto-fix lint issues
npm run lint:fix

# Format code
npm run format

# Type checking (if using TypeScript)
npm run type-check
```

Pre-commit Hooks

We use Husky for pre-commit hooks:

```
bash
```

```
# Hooks will automatically:
- Run linting
- Format code
- Run tests
- Check for secrets in code
```

Environment-Specific Setup

Development

- Use `.env.development`
- Hot reload enabled
- Debug logging on
- Local database

Staging

- Use `.env.staging`
- Production build
- Test with production-like data
- Remote database

Production

- Use `.env.production`
- Optimized build
- Error tracking enabled
- Production database

Security Checklist

Before deploying:

- [ ] All API keys in `.env` (not hardcoded)
- [ ] `.env` added to `.gitignore`
- [ ] Secure passwords for database

- [ ] HTTPS enabled for production
- [ ] CORS configured properly
- [ ] Rate limiting enabled
- [ ] Input validation on all endpoints
- [ ] Dependencies updated (no critical vulnerabilities)
- [ ] Secrets rotated from development keys

Resources

Official Documentation

- Node.js Docs
- PostgreSQL Docs
- Redis Docs
- Anthropic Claude Docs

Learning Resources

- Docker Tutorial
- Git Handbook
- REST API Best Practices

Community

- Project Discord/Slack
- GitHub Discussions
- Stack Overflow tag: `ai-gaming-companion`

Last Updated: November 2025
Maintainer: Your Name
Questions? Open an issue or contact the team